

Adora — GPU Memory: Host-Resident Column Batching

As-built notes · Ivan Milic · 2026-09-20 · session summary

The batched GPU synth (`synth_cube_map(..., batch=N)`, see [adora_9_eos_integration.pdf](#)) ran a 768^2 cube but died on larger ones. Cause: `batch` bounded the opacity intermediate but *not* the input residency. This note fixes that so GPU memory is bounded by `batch` regardless of cube size. Everything runs on the GPU by default (the `adora-develop` JAX is the CUDA build; `jax.default_backend()=="gpu"`).

1. The bug

The flatten uploaded the **whole** cube to the GPU before the chunk loop:

```
flat = [jnp.asarray(cube[k]).reshape(-1, nz) for k in keys]    # all columns, all keys -> GPU
```

`batch` then sliced arrays that were *already* on the device, so it capped the per-chunk opacity ($\sim \text{batch} \cdot n_\lambda \cdot n_z \cdot 4 \cdot 4 \cdot 8$ B) but left the full input cube resident. Rough resident inputs (7 keys, $n_z=121$, fp64): $768^2 \rightarrow \sim 4$ GB (fits), $1536^2 \rightarrow \sim 16$ GB (does not, alongside compute + output).

2. The fix ([synth3d.py](#), `synth_cube_map`)

Keep `flat` on the **host** (numpy); move only the current chunk to the GPU and pull its intensity straight back:

```
shp = cube["T"].shape                # read shape without uploading anything
flat = [jnp.asarray(cube[k]).reshape(-1, nz) for k in keys]    # host (numpy)
...
for s in starts:
    sub = [jnp.asarray(a[s:s+batch]) for a in flat]            # host slice -> GPU (only this chunk)
    chunks.append(np.asarray(fn(addata, waves, dz, *sub)))      # intensity -> host, frees the GPU
I = np.concatenate(chunks, axis=0)
```

Peak GPU memory is now `batch` columns of inputs + their opacity, **independent of the total cube size**. The same warm `fn` is reused (one compile; a smaller final chunk recompiles once). The `batch=None` path is unchanged: jit uploads the whole cube and returns an on-device JAX array (still composes with `grad/jit`) — host batching is only for large forward runs.

3. Verified

Batched output is **bit-identical** to the whole-cube result ($\max|\Delta|=0$) for both the EOS (`pg`) and legacy `ne/nH` paths, including an uneven final chunk; `batch ≥ npix` falls back to the single call. (FAL-C toy cube, forward-only.)

4. Notes / next

Host RAM now holds the input cube and the concatenated fp64 output (~ 15 GB at 1536^2) — the new ceiling is host memory, not the GPU. The complementary $\sim 2\times$ lever is **float32**: x64 is currently hardcoded `True` in all modules; running `jax.enable_x64=False` globally would make the kernel constants (`jnp.eye(4)`, Planck) f32 too, halving memory — not wired as a switch yet, and the polarised DELO solve / optical-depth exponentials need a precision check first.